Understanding Automatically Constructed Context in LLM-Based Agents

Guochen Li
Faculty of Science
University of Helsinki
Helsinki, Finland
guochen.li@helsinki.fi

***Abstract*—Large language models (LLMs) are often used inside agent systems that combine tools, retrieval, and memory. In such systems, the model does not only receive the user query. The surrounding framework also builds an effective prompt by adding system instructions, placeholders, tool outputs, retrieved content, and conversation history.**

This paper studies automatically constructed context in a controlled LangChain-based LLM agent setup connected to the Grok API. The study logs the messages sent to the model before each LLM call and classifies them into five components: system_core, retrieval, memory, user, and tool. Four controlled conditions are compared: baseline, single-tool, multi-step tool, and retrieval-memory. Each condition contains three task instances, and each task is repeated four times.

The results show that prompt structure changes across the observed settings. The baseline condition already contains non-user context because the shared prompt template includes empty retrieval and memory placeholders. Calculator-tool use increases prompt size, but tool text remains small because the calculator returns short outputs. Retrieval and memory change the structure more strongly when these fields are filled with manually provided text. These findings suggest that automatically constructed context should be treated as part of agent system design.

***Index Terms*—LLM agents, prompt construction, retrieval-augmented generation, tool integration, context analysis, agent architectures**

I. Introduction

Large language models (LLMs) are increasingly used in agent systems that combine tools, retrieval modules, memory mechanisms, and other external components [1]–[3]. In these systems, the model input is not limited to the user message. The surrounding framework may construct an effective prompt that contains system instructions, user input, tool outputs, retrieved content, memory content, and framework-related placeholders [4], [5], [11], [12]. In this paper, this full set of messages is called automatically constructed context.

Automatically constructed context is important for AI engineering because it affects agent behavior, transparency, cost, and debugging. A model may respond not only to the visible user question, but also to content added by the framework, a tool, a retrieval module, or a memory field. However, users and developers may not always see these prompt components.

Prior work has studied tool use, retrieval-augmented generation, and memory mechanisms in LLM-based agents [4]–[7], [9], [10]. These studies often focus on capability, answer quality, or task performance. Less attention has been given to the internal structure of the effective prompt itself. This leads to a practical engineering question: what does the model actually receive when an agent framework constructs the prompt?

This paper studies automatically constructed context in a controlled LangChain-based agent setup. The study does not try to improve task accuracy or compare answer quality. Instead, it examines the prompt sent to the model before each LLM call. The experiment compares four controlled conditions: baseline, single-tool, multi-step tool, and retrieval-memory. The goal is not to describe all possible LLM-agent systems, but to make prompt construction visible in a small and reproducible setting.

The paper addresses the following research questions:

- *RQ1: What are the structural components of automatically constructed context in the observed LLM-based agent setup?*
- *RQ2: How do calculator-tool use, manually filled retrieval, and manually filled memory change context composition across the controlled agent configurations?*
- *RQ3: What are the AI engineering implications of these structural changes for prompt growth, transparency, and system design?*

To answer these questions, the study logs the messages sent to the LLM at each invocation and divides the logged context into five components: system_core, retrieval, memory, user, and tool. The analysis uses character count to compare prompt size and component ratios. Each condition contains three task instances, and each task is repeated four times, giving 12 runs per condition and 48 runs in total.

This paper makes three contributions. First, it provides a message-level way to inspect automatically constructed context. Second, it compares prompt composition across four controlled agent settings. Third, it shows that different context sources change prompt structure in different ways within this setup.

The remainder of this paper is organized as follows. Section II reviews background and related work. Section III describes the method. Section IV presents the results. Section V discusses the findings. Section VI explains threats to validity. Section VII concludes the paper.

II. Background and Related Work

A. Agentic LLM Systems

Agentic LLM systems use a language model as part of a larger software system. In these systems, the model may call tools, use retrieved documents, keep conversation history, or work with other agents [2], [3], [11]. The model input is therefore usually not only the user message. The framework may build a structured prompt that contains role instructions, user input, tool traces, retrieved passages, memory content, and template fields [4], [5], [11].

This point is central to the present study. If the framework adds content before the model call, then the prompt should be studied as a system artifact. It is shaped by framework design, prompt templates, and agent configuration, not only by the user.

B. Tools, Retrieval, and Memory as Context Sources

Tool use is one common feature of agentic LLM systems. Prior work has shown that LLMs can use external tools to support reasoning and action, such as calculator calls, search, or other function calls [4], [5], [14]. When a tool output is inserted into a later model call, it becomes part of the model context.

Retrieval is another common source of context. In retrieval-augmented generation, external documents are selected and added to the prompt so that the model can use extra information [6], [7]. Memory mechanisms can also add previous dialogue, summaries, or stored records into later prompts [9], [10]. In this paper, long-horizon interaction refers to an interaction in which earlier dialogue or stored records can affect later model responses.

These sources affect prompts in different ways. Tools often add execution traces or results. Retrieval adds external documents. Memory adds earlier interaction content. These additions may be useful, but they also make the final prompt harder to inspect.

C. Research Gap

Existing research has studied whether tools, retrieval, and memory improve agent capability or answer quality [4]–[7], [9], [10]. These directions are important, but they do not fully answer a different engineering question: what does the model actually receive when the framework constructs the prompt?

The present study addresses this gap by analyzing prompt composition directly. It does not evaluate answer quality. Instead, it measures how tool use, retrieval, and memory change the structure of the effective prompt in one controlled LangChain-based setup.

III. Methodology

A. Research Design

This study uses a controlled empirical design. It does not aim to improve task accuracy or compare answer quality. It focuses on how different agent settings change the prompt that is sent to the LLM.

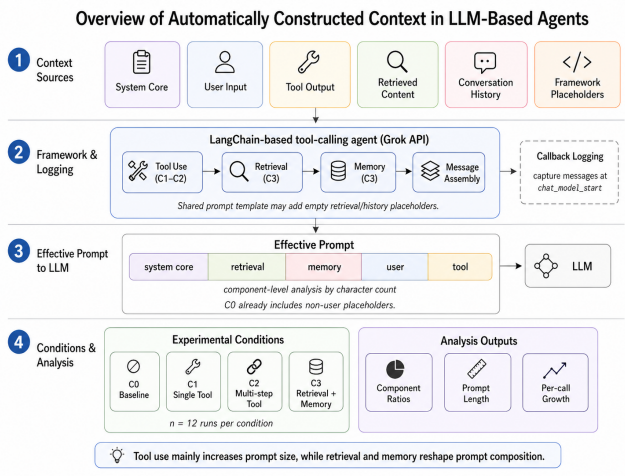

Fig. 1. Overview of automatically constructed context logging and analysis in the controlled LangChain-based agent setup.

The main object of analysis is automatically constructed context, defined as the full set of messages that the framework sends to the model before each LLM call. The experiment compares four settings of the same LangChain-based agent. These settings differ in their use of tools, retrieved content, and conversation history. All settings use the same logging and analysis procedure.

This study should be understood as a controlled exploratory experiment. The goal is not to represent all possible agent workflows, but to make effective prompt construction observable under comparable settings. Therefore, the reported ratios describe this experimental setup rather than universal proportions in all LLM-based agents.

B. System Instrumentation

The experiment uses a tool-calling agent implemented with LangChain [15] and connected to the Grok LLM API. The agent was instrumented so that the prompt sent to the model could be recorded before each model call. Fig. 1 summarizes the agent setup, context sources, logging point, and analysis outputs.

In this paper, *context*, *effective prompt*, and *effective input* refer to the same object: the full set of messages presented to the LLM at one invocation. A *message* means one role-labelled unit, such as a system message, human message, assistant message, or tool message.

A custom callback handler was used to intercept each `chat_model_start` event. For each model invocation, the logger stored the message type, message content, run identifiers, and task metadata in JSONL format. This made it possible to reconstruct the context of each LLM call, including calls inside tool-based workflows.

All four conditions used the same prompt template. The template contained a core system instruction, a retrieval placeholder, a conversation-history placeholder, the user input, and an agent scratchpad placeholder. Therefore, the main differences between conditions came from whether tool outputs, retrieved content, or history content were filled in.

C. Context Component Classification

Each captured message was assigned to a context component using deterministic rules based on message type and content. The study uses six categories: system_core, retrieval, memory, user, tool, and framework/scratchpad/other.

System messages beginning with "Retrieved context (if any):" were assigned to retrieval. System messages beginning with "Conversation history (if any):" were assigned to memory. Other system messages were assigned to system_core. Human messages were assigned to user, and tool messages were assigned to tool. Assistant messages containing tool-call metadata or scratchpad markers were assigned to framework/scratchpad/other.

The retrieval and memory categories include both filled content and empty placeholders. This is important for C0, where no retrieved documents or conversation history were provided, but the shared prompt template still contained empty retrieval and history fields. In the final aggregated results, framework/scratchpad/other did not add substantial character content after retrieval and history placeholders were assigned to their own categories. Therefore, the main results table reports five core components: system_core, retrieval, memory, user, and tool.

The study uses character length to measure context size. Character count is not the same as token count, but it is stable and easy to reproduce. It is used here to compare prompt structure across conditions, not to calculate exact API cost.

D. Experimental Conditions

The experiment defines four conditions. Each condition is designed to show a different source of prompt construction.

- **C0 (Baseline)** — no tool output, no retrieved document, and no conversation history are provided.
- **C1 (Single Tool)** — the agent uses one calculator-tool task.
- **C2 (Multi-step Tool)** — the agent uses chained calculator-tool tasks.
- **C3 (Retrieval + Memory)** — the prompt includes manually provided retrieved notes and conversation history.

The tool conditions use a simple calculator tool. The tool receives an arithmetic expression and returns the calculated result as text. This tool was selected because its behavior is easy to check and its output is deterministic.

The retrieval-memory condition uses manually written retrieved notes and manually written conversation history. These fields are inserted through the prompt template. They are not produced by a live retrieval system or a live memory store. This design helps isolate the structural effect of filled retrieval and memory fields.

Table I shows the task patterns used in the experiment.

To make the logged context more concrete, the following simplified example shows the structure of a C0 prompt. The actual logs were stored as role-labelled messages, but the example is shortened for space:

System_core: You are a helpful assistant.
Retrieval: Retrieved context (if any):
Memory: Conversation history (if any):
User: Explain what Fitts' law measures in 2–3 sentences.

TABLE I
EXAMPLE TASK PATTERNS USED IN THE EXPERIMENT

Condition	Example task pattern
C0	Explain what Fitts' law measures in 2–3 sentences.
C1	Use the calculator tool to compute one arithmetic expression, then explain the result briefly.
C2	Use the calculator tool for several arithmetic steps, then compare or summarize the results.
C3	Answer a Fitts' law question using manually provided retrieved notes and conversation history.

In C3, the same template was used, but the retrieval and memory fields were filled with manually provided notes and prior conversation history. This explains why C3 had a larger retrieval and memory share than C0.

Each condition contains three task instances. Each task instance is repeated four times. This gives 12 runs per condition. Across the four conditions, the experiment produces 48 runs in total. The task instances follow the same pattern inside each condition, but the wording, arithmetic values, or inserted context content may differ.

E. Analysis Procedure

All logs were analyzed using the same procedure. The analysis read the JSONL log files, extracted all `chat_model_start` events, classified each message into a predefined component, and counted the character length of each component. Component ratios were then calculated for each run.

The analysis uses two levels of summary. Run-level summaries compare the four conditions. Call-level traces examine how prompt content changes inside one selected multi-step tool run. The analysis separates total prompt growth from structural redistribution, where the largest prompt component shifts from one source to another.

IV. RESULTS

The results show that the structure of automatically constructed context changes across the four controlled conditions. Table II reports the mean component ratios. Fig. 2 shows the same ratios as a stacked bar chart. Fig. 3 shows the mean total prompt size in characters.

The results show three main patterns in this experimental setup. The baseline condition already contains non-user context because of empty template placeholders. The calculator-tool conditions increase prompt size, but tool outputs remain a small part of the prompt because the calculator returns short textual outputs. The retrieval-memory condition changes the structure more strongly because the retrieval and memory fields are filled with manually provided text.

A. Finding 1: The Baseline Prompt Already Contains Non-User Context

In C0, the agent did not receive retrieved documents or conversation history. The context was mainly made of sys-

TABLE II
MEAN CONTEXT COMPONENT RATIOS ACROSS CONDITIONS. C0 = BASELINE, C1 = SINGLE TOOL, C2 = MULTI-STEP TOOL, AND C3 = RETRIEVAL + MEMORY.

Component	C0	C1	C2	C3
system_core	30.3%	30.2%	20.6%	9.1%
retrieval	16.6%	16.6%	11.3%	38.7%
memory	18.4%	18.4%	12.5%	34.5%
user	34.6%	33.8%	53.4%	17.6%
tool	0.0%	1.1%	2.1%	0.0%

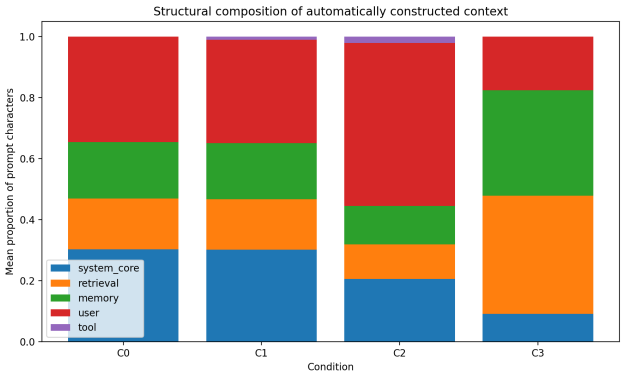

Fig. 2. Structural composition of automatically constructed context across the four experimental conditions. C0 = Baseline, C1 = Single Tool, C2 = Multi-step Tool, and C3 = Retrieval + Memory.

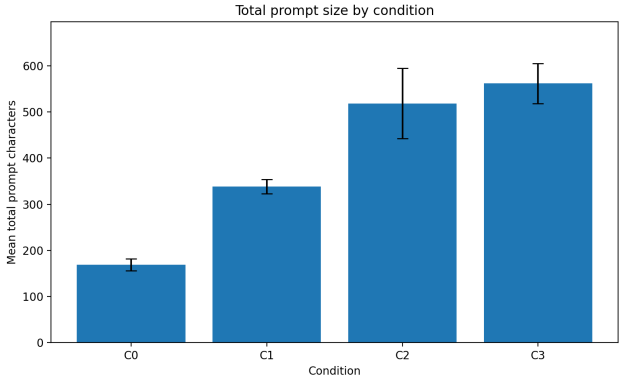

Fig. 3. Mean total prompt length in characters across the four experimental conditions. Error bars indicate standard deviation across 12 runs per condition.

tem_core content and user input. System_core contributed 30.3% of the prompt, and user input contributed 34.6%.

However, C0 also contained retrieval-related and memory-related content. Retrieval accounted for 16.6%, and memory accounted for 18.4%. These parts were not filled retrieved documents or real conversation history. They were empty placeholders from the shared prompt template. Together, these two placeholder components accounted for about 35.0% of the prompt.

The mean total prompt size in C0 was 169.0 characters, with a standard deviation of 12.6 characters. This result shows that even the baseline setting was not only a user query and a core system message. The framework template already added visible structure to the prompt.

B. Finding 2: Calculator-Tool Use Increases Prompt Size, But Tool Text Remains Small in This Setting

C1 added one calculator-tool task. The tool component appeared in the prompt and accounted for 1.1% of the total context. The rest of the prompt stayed close to the baseline structure.

The mean total prompt size in C1 was 338.3 characters, with a standard deviation of 15.3 characters. This is about twice the size of C0. C2 used chained calculator-tool tasks. In this condition, the tool component increased to 2.1%, while user input became the largest component at 53.4%, mainly because the multi-step tool tasks used longer instructions.

The mean total prompt size in C2 was 518.4 characters, with a standard deviation of 76.2 characters. This condition also had the largest variation in prompt size, which is consistent with multi-step tool use.

These results should be read in the context of the calculator tool used in this study. The calculator returned short textual results. Therefore, the tool component remained small. This result should not be interpreted as evidence that tool outputs are generally small in all agent systems. A different tool returning long documents, search results, logs, API responses, or structured data could produce a much larger tool component.

C. Finding 3: Manually Filled Retrieval and Memory Fields Change the Main Structure of the Prompt

C3 produced the largest structural change. Retrieval accounted for 38.7% of the prompt, and memory accounted for 34.5%. Together, these two components made up 73.2% of the total prompt.

This result differs from C0 because retrieval and memory were empty placeholders in C0 but manually filled with notes and conversation history in C3. Therefore, C3 should be interpreted as a controlled retrieval-memory injection result. The high retrieval and memory ratio does not mean that these components will always dominate real agent prompts. It shows that filled retrieval and memory fields can become the main source of prompt content when longer text blocks are inserted.

The mean total prompt size in C3 was 561.7 characters, with a standard deviation of 43.3 characters. C3 had the largest mean prompt size among the four conditions. At the same time, system_core dropped to 9.1%, and user input dropped to 17.6%. The prompt was therefore mainly shaped by the filled retrieval and memory fields in this condition.

D. Per-Call Growth in the Multi-Step Tool Condition

Fig. 4 shows one selected C2 run at the call level. The figure shows how the prompt changes between successive LLM calls inside a multi-step tool workflow.

In this example, Call 2 contains more prompt content than Call 1. The tool component also appears in Call 2. This pattern shows that later calls can include content produced earlier in the workflow. However, the tool part remains smaller than the user and system-related parts in this example.

This call-level result supports the condition-level result for C2. Tool use adds content to later prompts, but the calculator-tool output does not become the main part of the prompt.

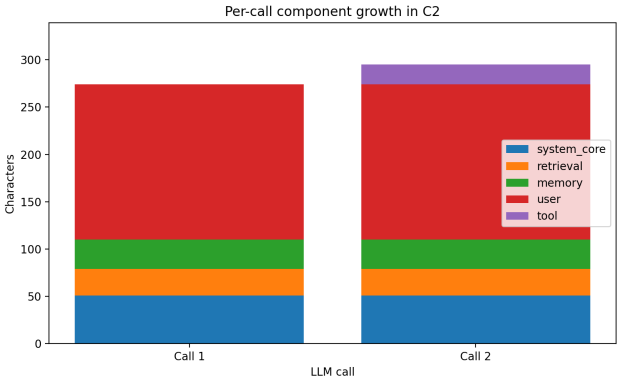

Fig. 4. Per-call component growth in the multi-step tool condition (C2). The x-axis represents successive LLM calls within one selected C2 run.

E. Cross-Condition Comparison

The four conditions show two types of prompt change. C1 and C2 mainly show prompt growth, while C3 shows structural redistribution. C2 and C3 both increase prompt size, but they do so in different ways. C2 grows mainly through longer user instructions and tool-related interaction. C3 changes the main source of prompt content because retrieval and memory fields are filled with longer text.

C2 has the largest standard deviation in total prompt size, suggesting that multi-step tool workflows can produce more variation across runs. C3 has the largest mean prompt size, but its structure is more stable because the manually filled retrieval and memory fields follow a fixed template. These comparisons should not be read as universal ratios for all agent systems.

V. Discussion

A. Answer to RQ1: Structural Components of Automatically Constructed Context

The results answer RQ1 by showing that automatically constructed context is not only made of a user query and a system message in the observed setup. The effective prompt contained several parts: system_core, retrieval, memory, user, and tool. These parts came from both the user and the agent framework.

This finding is important because C0 did not receive retrieved documents or conversation history, but it still contained retrieval-related and memory-related placeholders from the shared prompt template. Message-level logging made this structure visible. If the analysis only looked at the visible user question and final answer, this framework-level structure would not be observable.

B. Answer to RQ2: Effects of Tools, Retrieval, and Memory

The results answer RQ2 by showing that calculator-tool use, manually filled retrieval, and manually filled memory changed the prompt in different ways. In C1 and C2, the calculator tool added content to later prompts, but the tool component stayed small because the calculator returned short text outputs. A tool returning long search results, documents, logs, API responses, or structured data could produce a larger tool component.

C3 behaved differently. Retrieval and memory were manually filled with notes and conversation history, and these fields became the largest parts of the prompt. This shows structural redistribution. The prompt did not only become longer; its main source of content changed. The exact ratios should be understood as results of this controlled setup, not as general proportions for all LLM-based agents.

C. Answer to RQ3: AI Engineering Implications

The results answer RQ3 by showing that prompt construction should be treated as an engineering concern. Even in this small controlled LangChain setup, the prompt was not only a text field written by the user. It was also shaped by prompt templates, tool traces, retrieved content, and memory fields.

This matters for prompt growth, transparency, and control. A longer prompt can affect token use and cost. A prompt with large retrieval or memory fields may also make system behavior depend strongly on how these fields are selected and formatted. Therefore, developers should inspect the full effective prompt rather than only the visible user query and final answer.

D. Relation to Prior Work

The results do not conflict with prior work on tools, retrieval, and memory. Prior work has shown that tools can support acting and problem solving [4], [5], [14], retrieval can support factual grounding [6], [7], and memory can support longer interactions [9], [10].

This study adds a different view. It does not evaluate whether these mechanisms improve answer quality. Instead, it examines how they change the input that the model receives. The findings also connect to work on long-context use [8]. Prompt growth should not only be measured by length. It should also be described by source and structure.

VI. Threats to Validity

Several limitations should be considered when interpreting the results. Overall, this study should be read as a controlled exploratory experiment rather than a general benchmark of all LLM-agent frameworks.

First, this study used character count as the main measure of prompt size. Character count is stable and easy to reproduce, but it is not the same as token count. The reported counts should therefore be understood as structural measures of prompt size, not exact measures of API cost.

Second, the experiment used a LangChain-based agent connected to the Grok LLM API. Other models, providers, or agent frameworks may use different message formats, prompt templates, tool-calling formats, or memory structures. The exact component ratios may therefore not transfer directly to other systems.

Third, all four conditions used the same shared prompt template. This helped keep the framework structure constant across conditions, but the results also depend on this template. A different template could change the balance between system_core, retrieval, memory, user, and tool components.

Fourth, the experiment used a small and controlled task set. The baseline and retrieval-memory conditions used Fitts' law explanation prompts, while the tool conditions used calculator-based arithmetic prompts. More complex tasks may produce longer tool outputs, more retrieval content, or more varied memory use.

Fifth, the calculator tool returned short arithmetic results as text. This explains why the tool component remained small. A different tool returning long search results, documents, logs, API responses, or structured data could produce a much larger tool component.

Sixth, retrieved notes and conversation history in C3 were manually provided through the prompt template. They were not generated by a live retrieval system or memory store. Real systems may produce content with different length, format, relevance, and noise.

Finally, context components were assigned using deterministic rules based on message type and content patterns. This method was consistent across all runs and suitable for the controlled setup because the prompt template used explicit retrieval and memory prefixes. However, it may miss ambiguous cases in more complex prompts. Future work could compare this rule-based approach with manual coding or inter-rater agreement.

VII. Conclusion

This paper studied automatically constructed context in a controlled LangChain-based LLM agent setup. The study logged the messages sent to the model before each LLM call and divided the logged context into five components: system_core, retrieval, memory, user, and tool.

The results show that different agent settings changed prompt structure in different ways within this setup. The baseline setting already contained non-user context because the shared prompt template included empty retrieval and memory placeholders. Calculator-tool use increased prompt size, but tool text remained small because the calculator returned short outputs. Manually filled retrieval and memory fields changed the structure more strongly because they added longer text blocks.

The main lesson is that prompt growth is not only about length. The source of the prompt content also matters. Automatically constructed context should therefore be treated as a design-level part of agent systems, not as a hidden detail inside the framework.

These findings provide a small but concrete example of how developers can inspect agent prompts and reason about context sources. Future work should extend this analysis with token-level measurements, more agent frameworks, live retrieval and memory systems, and more realistic tool workflows.

References

[1] M. A. K. Raiaan, M. S. H. Mukta, K. Fatema, N. M. Fahad, S. Sakib, M. M. J. Mim, J. Ahmad, M. E. Ali, and S. Azam, "A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges," *IEEE Access*, vol. 12, pp. 26839–26874, 2024, doi: 10.1109/ACCESS.2024.3365742.

[2] D. B. Acharya, K. Kuppan, and B. Divya, "Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey," *IEEE Access*, vol. 13, pp. 18912–18936, 2025, doi: 10.1109/ACCESS.2025.3532853.

[3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, Art. no. 186345, 2024, doi: 10.1007/s11704-024-40231-1.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.

[5] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[7] H. Yu, Z. Zhang, X. Li, and B. Li, "Evaluation of Retrieval-Augmented Generation: A Survey," arXiv:2405.07437, 2024.

[8] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[9] Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen, J. Zhu, Z. Hu, and J.-R. Wen, "A Survey on the Memory Mechanism of Large Language Model based Agents," *ACM Trans. Inf. Syst.*, 2025, doi: 10.1145/3748302.

[10] C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.

[11] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," in *Proc. Conf. Language Modeling (COLM)*, 2024.

[12] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan A, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[13] H. Strobelt, A. Webson, V. Sanh, B. Hoover, J. Beyer, H. Pfister, and A. M. Rush, "Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with Large Language Models," *IEEE Trans. Vis. Comput. Graph.*, vol. 29, no. 1, pp. 1146–1156, Jan. 2023, doi: 10.1109/TVCG.2022.3209479.

[14] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[15] LangChain, "LangChain Documentation," 2026. [Online]. Available: https://python.langchain.com/